

Abstract

This is the report for Exercise 1. In this report we showcase the steps taken to compile the OSU Benchmark and run said benchmarks on two collective operations on the ORFEO cluster, as described in the [GitHub repository](#).

1 OSU Benchmarks

The OSU Micro-Benchmarks (OMB) are a collection of benchmarks designed to measure the performance of communication operations in MPI. They provide a standardized way to evaluate metrics such as latency and bandwidth for different message sizes and numbers of processes.

For this exercise, two collective communication operations are considered: MPI_Bcast and MPI_Reduce. The corresponding OSU benchmarks are run on the ORFEO cluster under different configurations to evaluate their communication performance.

1.1 MPI_Bcast

MPI_Bcast is a collective operation that broadcasts a message from a designated root process to all other processes in the communicator. The data provided by the root is therefore replicated across all processes.

The OSU broadcast benchmark measures the time required to perform this operation for different message sizes and numbers of processes. This allows us to evaluate how the broadcast performance changes as the amount of data and the number of participating processes increase.

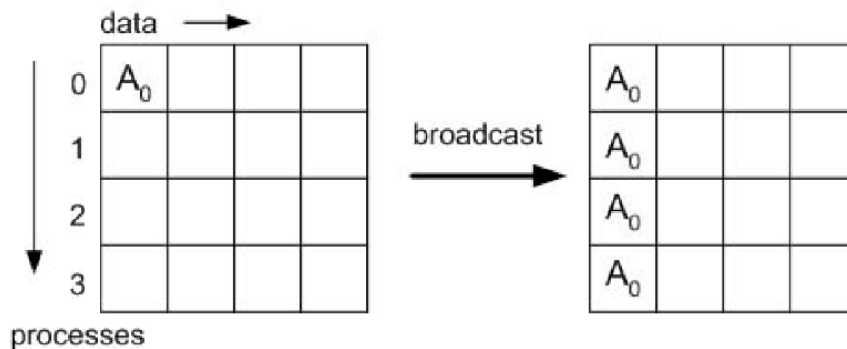


Figure 1: Graphical representation of MPI_Bcast.

1.2 MPI_Reduce

MPI_Reduce is a collective operation that combines data from all processes using a specified reduction operation, such as a sum, and stores the resulting value on a designated root process.

The OSU reduce benchmark measures the time required to perform this operation for different message sizes and numbers of processes. This allows us to evaluate the communication and reduction overhead as the number of processes and the amount of data increase.

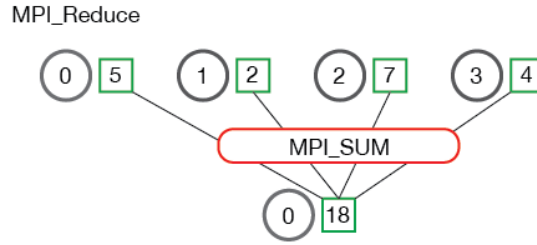


Figure 2: Graphical representation of MPI_Reduce.

2 Compiling

The OSU Micro-Benchmarks are compiled directly on the ORFEO cluster using the MPI implementation that will also be used to run the benchmarks. This ensures that the generated binaries are compatible with the cluster environment and the selected MPI implementation.

The compilation is performed through a Slurm job on the THIN partition. The appropriate MPI module is loaded before compiling, after which the OSU Micro-Benchmarks source code is downloaded and configured using the MPI compiler wrappers `mpicc` and `mpicxx`:

```
./configure CC=mpicc CXX=mpicxx --prefix="${OSU_PATH}"
make
make install
```

The resulting binaries are installed in a custom directory specified by `OSU_PATH`, allowing them to be used when running the benchmarks without recompiling them for each experiment. Separating the compilation step from the benchmark runs also avoids unnecessarily using compute resources for compilation. Scripts are provided in the repository for ease of use.

3 Benchmark results

The benchmarks were executed on the ORFEO cluster using different numbers of MPI processes and different collective communication algorithms. The three algorithms considered for each collective operation are:

- **MPI_Bcast:**
 - **Basic linear:** the root process sends the message directly to each process sequentially.
 - **Pipeline:** the message is divided into segments that are forwarded between processes, allowing communication to overlap.
 - **Binomial tree:** processes are organized in a tree structure, allowing the message to reach all processes in approximately $\log_2(P)$ communication steps.
- **MPI_Reduce:**
 - **Linear:** processes send their data directly towards the root, which performs the reduction.
 - **Pipeline:** data is divided into segments and reduction is performed progressively as segments move through the processes.
 - **In-order binary:** processes are organized as a binary tree, with partial results combined at each level before reaching the root.

For each configuration, the benchmark was executed 10 times and the measured results were collected for different message sizes. The number of MPI processes was varied from 2 to the maximum number supported by the allocated nodes, in steps of two processes.

3.1 Point-to-Point latency

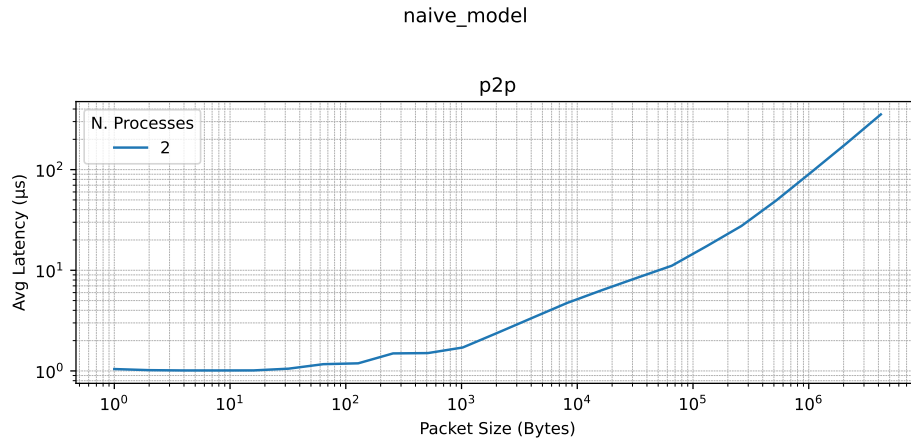
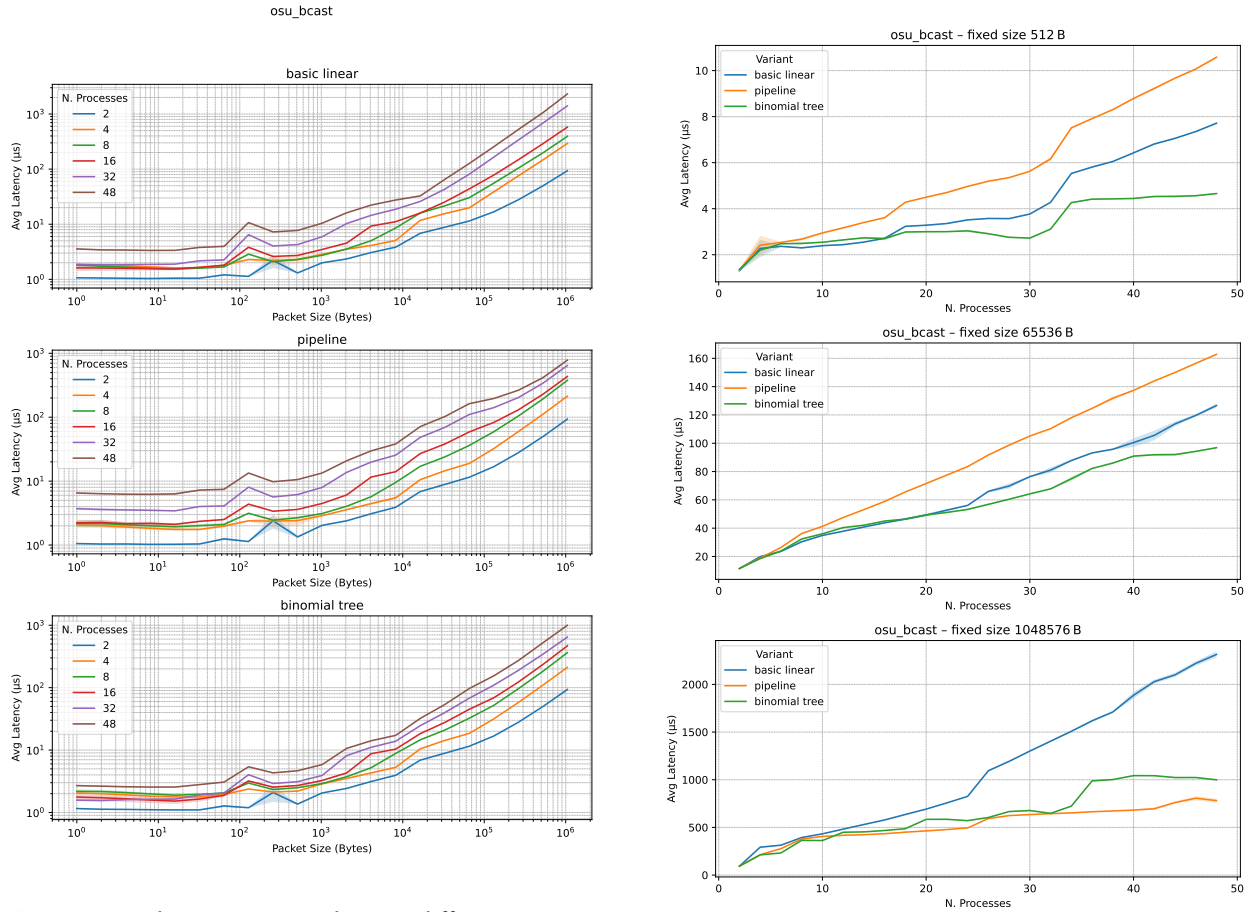


Figure 3: Point-to-point latency measured with two MPI processes.

The point-to-point latency provides a baseline for comparison with the collective communication results. As expected, the latency remains approximately constant for small messages, up to around 128 bytes, before increasing progressively with message size. In the log-log plot, this produces the characteristic curved shape visible in Figure 3. The latency is approximately $1\mu\text{s}$ for the smallest messages and increases to $400\mu\text{s}$ as the message size approaches the maximum tested value of 4 MB.

3.2 MPI_Bcast



(a) MPI_Bcast latency measured across different MPI processes amounts and collective algorithms.

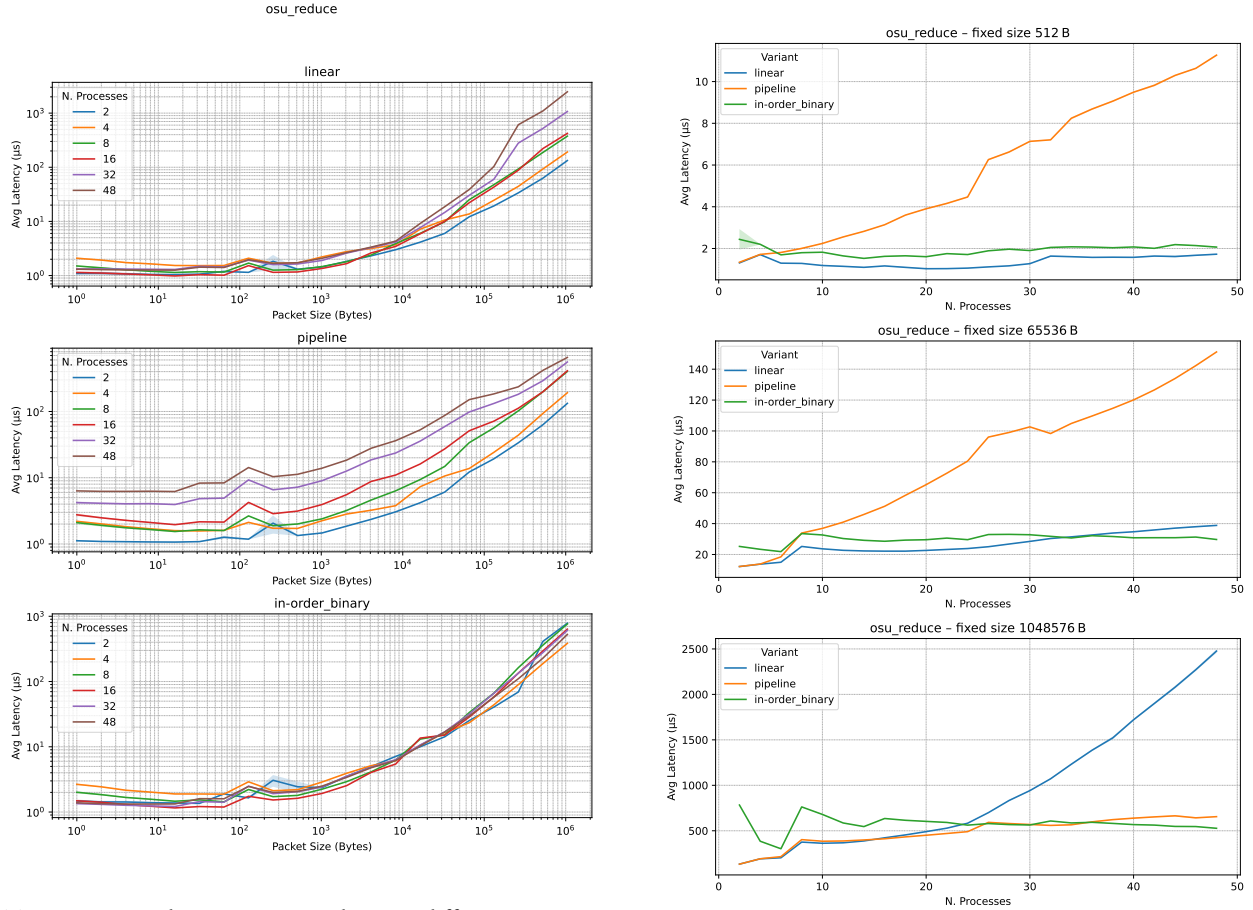
(b) MPI_Bcast latency across different packet sizes.

Figure 4: Plots for MPI_Bcast.

The results for MPI_Bcast show clear differences in how the three collective algorithms scale with the number of processes and the message size. The basic linear algorithm exhibits the expected linear increase in latency as the number of processes grows. The pipeline algorithm follows a similar trend for small messages, but its performance improves considerably for larger messages and higher numbers of processes. The binomial tree algorithm scales better with the number of processes overall, although for large messages and a high number of processes, the pipeline algorithm achieves slightly lower latency.

The fixed-message-size plots provide a more detailed comparison between the algorithms. For the smallest message size, 512 bytes, all three algorithms exhibit a similar scaling shape as the number of processes increases, although they operate at slightly different latency levels. In this case, the binomial tree algorithm has the lowest latency, followed by the basic linear algorithm, while the pipeline algorithm performs slightly worse. As the message size increases to 64 KB and 1 MB, the relative performance changes: the pipeline algorithm becomes increasingly advantageous as throughput becomes more important, eventually achieving the lowest latency for large messages and high process counts. The binomial tree algorithm maintains good scaling and remains competitive, while the basic linear algorithm becomes increasingly affected by the growing number of processes.

3.3 MPI_Reduce



(a) MPI_Reduce latency measured across different MPI processes amounts and collective algorithms.

(b) MPI_Reduce latency across different packet sizes.

Figure 5: Plots for MPI_Reduce.

The MPI_Reduce results show a different performance trade-off between the three algorithms. The linear algorithm achieves the lowest latency for small messages and is particularly effective at 512 bytes. However, its latency increases significantly as the message size and number of processes grow, resulting in poor scaling for larger messages combined with high process counts.

The pipeline algorithm has the highest latency for small messages and its latency increases considerably with the number of processes. However, its performance improves relative to the other algorithms as the message size increases. For large messages, the pipeline becomes the most efficient algorithm for smaller numbers of processes, benefiting from the ability to overlap communication and reduction across message segments.

The in-order binary algorithm provides more consistent scaling across the tested configurations. Its tree-based structure limits the increase in communication steps as the number of processes grows, making it competitive across different message sizes. For large numbers of processes, it generally achieves the lowest latency, while for very small messages the linear algorithm remains more efficient.

3.4 Normalized Latency

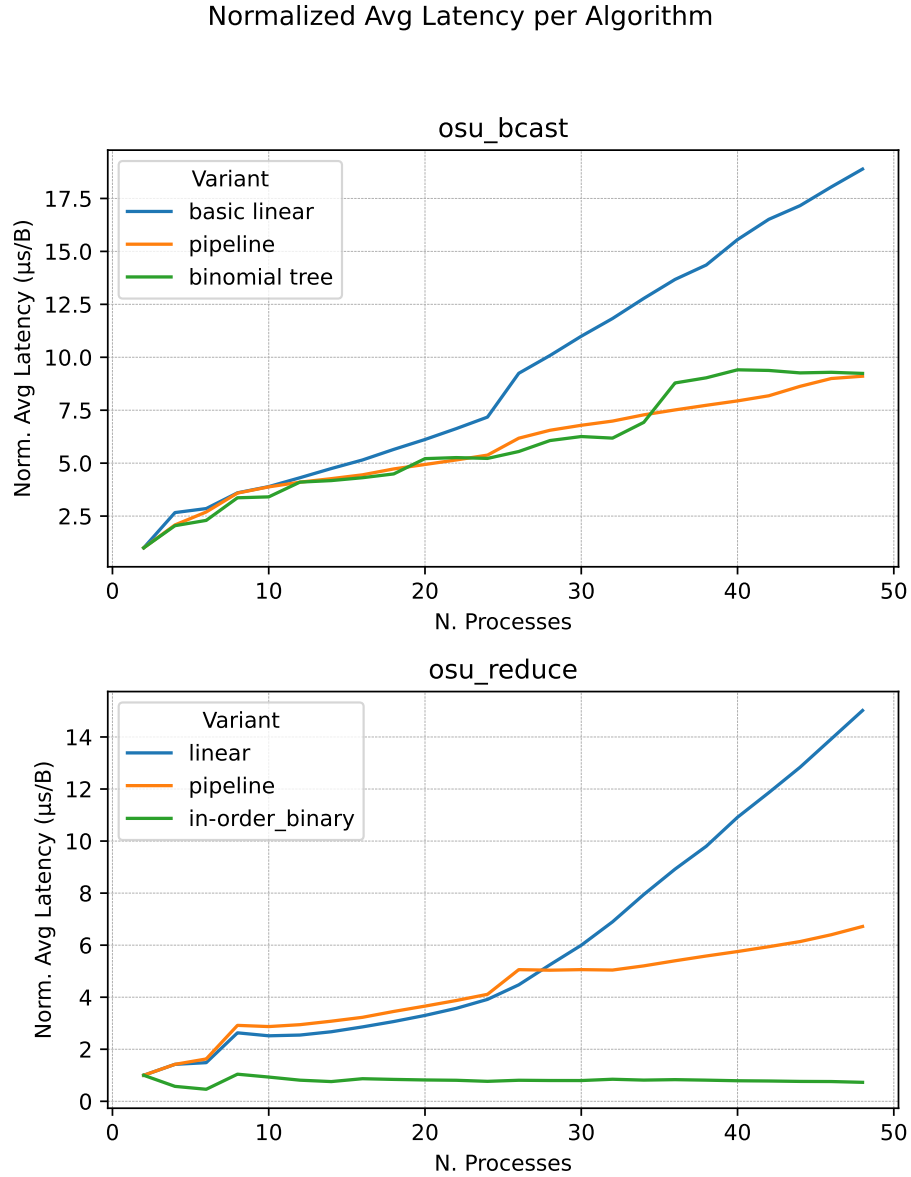


Figure 6: Normalized latency for MPI_Bcast and MPI_Reduce.

The normalized latency highlights how the communication cost scales with the number of processes relative to the message size. Since the benchmarks were executed on two nodes with 24 processes per node, configurations with up to 24 processes can be accommodated on a single node, while configurations with more than 24 processes require both nodes.

For MPI_Bcast, the basic linear algorithm shows an approximately linear increase in normalized latency, with a noticeable increase around 24 processes. It grows from approximately $1\mu\text{s}/B$ to $18\mu\text{s}/B$. The pipeline algorithm scales more slowly and does not exhibit the same increase at 24 processes, reaching approximately $10\mu\text{s}/B$. The binomial tree algorithm shows a more irregular behavior, with several smaller fluctuations, but follows an overall trend similar to the pipeline algorithm and reaches a comparable value.

For MPI_Reduce, the linear algorithm again shows an approximately linear increase, with a steeper growth after 24 processes, reaching approximately $15\mu\text{s}/B$. The pipeline algorithm follows a similar trend up to 24 processes, where a pronounced increase is observed, after which the normalized latency remains relatively stable at approximately $6\mu\text{s}/B$. The in-order binary algorithm exhibits the most stable behavior, remaining close to $1\mu\text{s}/B$ across the tested process counts.